SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	Mohan Kumar J	**Reg. No.:**	192424060
Section / Batch:	2024 [AI&DS]	**Date:**	25-08-2026

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

1. Bin Packing Approximation Algorithm

Write a pseudocode for an approximation algorithm such as First-Fit Decreasing to pack items into bins of fixed capacity. Clearly define inputs (item sizes, bin capacity) and output (set of bins with assigned items). Design an algorithm that first orders items and then places each item into the first suitable bin. Use loops and conditional checks effectively for sorting and bin assignment. Ensure the pseudocode is well-structured, readable, and logically organized. Present all major steps clearly in sequence. Handle all items correctly and produce a valid approximate packing solution.

Code of Conduct

I Mohan Kumar J (192424060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Mohan Kumar

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm Design	30%				
Use of Control Structures	20%				
Pseudocode Structure	10%				
Completeness	20%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Bin Packing Approximation Algorithm Using First-Fit Decreasing

1. Problem Understanding

Problem Statement

- The Bin Packing Problem is an optimization problem in which a set of items with different sizes must be packed into a minimum number of bins.
- Each bin has a fixed capacity, and the total size of items placed in any bin must not exceed that capacity.
- For this problem, we use the First-Fit Decreasing (FFD) approximation algorithm.

The algorithm works in two main stages:

1. Sort all items in **decreasing order of size**.
2. Place each item into the **first bin that has enough remaining capacity**.

If no existing bin can accommodate the item, a new bin is created.

Inputs

The algorithm requires:

- n – number of items.
- items[] – sizes of the items.
- capacity – maximum capacity of each bin.

Outputs

The algorithm produces:

- The number of bins used.
- The contents of each bin.
- The total size of items in each bin.
- The remaining capacity of each bin.

Constraints

The following constraints must be satisfied:

1. n >= 0
2. capacity > 0
3. Every item size must be positive.
4. No item can have a size greater than the bin capacity.

5. The total size of items in a bin must not exceed its capacity.
6. Every item must be assigned to exactly one bin.
7. Items should be processed in decreasing order.

2. Algorithm Design

First-Fit Decreasing Algorithm

- First-Fit Decreasing is an approximation algorithm for the Bin Packing Problem.
- The algorithm first sorts the items from largest to smallest. It then considers each item and searches the existing bins from the first bin onward.
- If the item fits into a bin, it is placed there immediately.
- If it does not fit into any existing bin, a new bin is opened.

Main Steps

1. Read the number of items.
2. Read the item sizes.
3. Read the bin capacity.
4. Validate the input.
5. Sort the items in decreasing order.
6. Create an empty list of bins.
7. Process every item one by one.
8. Check each existing bin.
9. Place the item into the first suitable bin.
10. If no bin is suitable, create a new bin.
11. Continue until all items are placed.
12. Display the final bin arrangement.

3. Pseudocode

```
BEGIN

    INPUT n
    INPUT items[1...n]
    INPUT capacity

    IF capacity <= 0 THEN
        PRINT "Invalid bin capacity"
        STOP
    END IF

    FOR each item in items DO
        IF item <= 0 OR item > capacity THEN
            PRINT "Invalid item size"
            STOP
        END IF
    END FOR

    SORT items in decreasing order

    bins ← empty list
    remaining ← empty list

    FOR each item in items DO

        placed ← FALSE

        FOR i ← 0 TO length(bins) - 1 DO

            IF remaining[i] >= item THEN

                ADD item to bins[i]
                remaining[i] ← remaining[i] - item

                placed ← TRUE
                BREAK

            END IF

        END FOR

        IF placed = FALSE THEN
```

```
                CREATE new bin
                ADD item to new bin

                ADD new bin to bins
                ADD (capacity - item) to remaining

            END IF

        END FOR

        PRINT "Number of bins:", length(bins)

        FOR i ← 0 TO length(bins) - 1 DO
            PRINT "Bin", i + 1
            PRINT "Items:", bins[i]
            PRINT "Used capacity:", capacity - remaining[i]
            PRINT "Remaining capacity:", remaining[i]
        END FOR

    END
```

4. Control Structures Used

The algorithm uses appropriate control structures to make the solution efficient and logically organized.

4.1 For Loop

- A for loop is used to process every item.
- FOR each item in items DO
- This ensures that every item is considered exactly once.

4.2 Nested For Loop

Another loop checks the existing bins.

FOR i ← 0 TO length(bins) - 1 DO

The bins are examined from the first bin to the last bin.

4.3 If Condition

The following condition determines whether an item can fit:

IF remaining[i] >= item THEN

If the condition is true, the item is placed in that bin.

4.4 Break

Once an item is placed in the first suitable bin, the search stops:

BREAK

This implements the **First-Fit** strategy.

4.5 Boolean Variable

The variable placed records whether the item has been successfully assigned.

placed ← FALSE

After placing the item:

placed ← TRUE

If it remains false after checking all bins, a new bin is created.

5. Detailed Working

Consider the following example.

Input

Number of items = 7

Items = 4, 8, 1, 4, 2, 7, 3

Bin capacity = 10

Step 1: Sort in Decreasing Order

Original items:

4, 8, 1, 4, 2, 7, 3

After decreasing-order sorting:

8, 7, 4, 4, 3, 2, 1

Step 2: Place Items

Item = 8

No bins exist, so create Bin 1.

Bin 1: [8]
Remaining = 2

Item = 7

It does not fit into Bin 1.

Create Bin 2.

Bin 1: [8] Remaining = 2
Bin 2: [7] Remaining = 3

Item = 4

It does not fit into Bin 1 or Bin 2.

Create Bin 3.

Bin 1: [8] Remaining = 2
Bin 2: [7] Remaining = 3
Bin 3: [4] Remaining = 6

Item = 4

It does not fit into Bin 1.

It does not fit into Bin 2.

It fits into Bin 3.

Bin 1: [8] Remaining = 2
Bin 2: [7] Remaining = 3
Bin 3: [4, 4] Remaining = 2

Item = 3

It does not fit into Bin 1.

It fits into Bin 2.

Bin 1: [8] Remaining = 2
Bin 2: [7, 3] Remaining = 0
Bin 3: [4, 4] Remaining = 2

Item = 2

It fits into Bin 1.

Bin 1: [8, 2] Remaining = 0
Bin 2: [7, 3] Remaining = 0
Bin 3: [4, 4] Remaining = 2

Item = 1

It fits into Bin 3.

Bin 1: [8, 2] Remaining = 0
Bin 2: [7, 3] Remaining = 0
Bin 3: [4, 4, 1] Remaining = 1

Final Packing

Bin 1: [8, 2]
Bin 2: [7, 3]
Bin 3: [4, 4, 1]

Therefore:

Number of bins used = 3

Every bin has a total size of at most 10.

6. Python Implementation

```
def first_fit_decreasing(items, capacity):

    # Validate capacity
    if capacity <= 0:
        print("Invalid bin capacity")
        return

    # Validate item sizes
    for item in items:
        if item <= 0 or item > capacity:
            print("Invalid item size:", item)
            return

    # Sort items in decreasing order
    items.sort(reverse=True)

    bins = []
    remaining = []

    # Place each item
    for item in items:

        placed = False

        # Check existing bins
        for i in range(len(bins)):

            if remaining[i] >= item:

                bins[i].append(item)
```

```
                remaining[i] -= item

                placed = True
                break

        # Create a new bin if item does not fit
        if not placed:

            bins.append([item])
            remaining.append(capacity - item)

    # Display result
    print("Number of bins used:", len(bins))
    print()

    for i in range(len(bins)):

        used = capacity - remaining[i]

        print("Bin", i + 1, ":", bins[i])
        print("Used capacity:", used)
        print("Remaining capacity:", remaining[i])
        print()


# Example input
items = [4, 8, 1, 4, 2, 7, 3]
capacity = 10

first_fit_decreasing(items, capacity)
```

7. Sample Input and Output

Input

7
4 8 1 4 2 7 3
10

The values represent:

7 → Number of items
4 8 1 4 2 7 3 → Item sizes
10 → Bin capacity

Output

Number of bins used: 3

Bin 1 : [8, 2]
Used capacity: 10
Remaining capacity: 0

Bin 2 : [7, 3]
Used capacity: 10
Remaining capacity: 0

Bin 3 : [4, 4, 1]
Used capacity: 9
Remaining capacity: 1

The exact order of bins can depend on the implementation, but the number of bins and valid packing should be consistent for this example.

8. Correctness of the Algorithm

The algorithm correctly produces a valid packing because every item is considered and assigned according to the bin capacity constraint.

Item Assignment

- Each item is processed exactly once.
- Therefore, no item is accidentally skipped.

Capacity Constraint

An item is placed into a bin only when:

remaining capacity >= item size

Therefore:

Total Size Of Items In Every Bin <= Bin Capacity

First-Fit Property

- The algorithm examines bins from the first bin onward.
- The item is placed into the first bin that can accommodate it.
- Therefore, the algorithm follows the First-Fit strategy.

Decreasing Order

- Before placement, the items are sorted in decreasing order.
- Therefore, the algorithm follows the First-Fit Decreasing (FFD) strategy.

New Bin Creation

- If no existing bin can hold the item, a new bin is created.
- Therefore, every item will eventually be placed as long as its size does not exceed the bin capacity.

9. Approximation and Complexity Analysis

The Bin Packing Problem is an NP-hard optimization problem. Finding the absolute minimum number of bins can be computationally expensive for large inputs. First-Fit Decreasing provides an efficient approximation approach.

Time Complexity

Let:

- **n = number of items**
- **b = number of bins**

Sorting requires:

$$\mathbf{O(n \log n)}$$

For each item, the algorithm may examine every existing bin.

Therefore, the placement phase requires approximately:

$$\mathbf{O(n \times b)}$$

Hence, the **worst-case time complexity** is:

$$\mathbf{O(n^2)}$$

Including sorting:

$$\mathbf{O(n \log n + n^2)}$$

which simplifies to:

$\mathbf{O(n^2)}$ in the worst case.

Space Complexity

The bins store every item exactly once.

Therefore, the space required is:

$$\mathbf{O(n)}$$

excluding the input storage.

10. Why First-Fit Decreasing Is an Approximation Algorithm

- The goal of Bin Packing is to minimize the number of bins.
- The FFD algorithm does not always guarantee the globally minimum number of bins.
- For example, a different arrangement of the same items may sometimes use fewer bins.
- However, FFD generally produces a good packing while requiring much less computational effort than an exact optimization algorithm.

A known theoretical guarantee for First-Fit Decreasing is:

$$\mathbf{FFD(I) \leq (11/9)\ OPT(I) + 6/9}$$

where:

- FFD(I) is the number of bins used by FFD.
- OPT(I) is the optimal minimum number of bins.

Thus, FFD provides a bounded approximation rather than attempting an exhaustive search for the optimal solution.

11. Handling Special Cases

The algorithm should correctly handle the following cases.

Case 1: Empty Item List

If:

items = []

then no bins are required.

Number of bins = 0

Case 2: One Item

If there is only one valid item, one bin is sufficient.

Example:

items = [5]
capacity = 10

Result:

Bin 1: [5]

Case 3: Item Equal to Capacity

An item exactly equal to the bin capacity can occupy an entire bin.

Example:

item = 10
capacity = 10

The remaining capacity becomes: 0

Case 4: Item Greater Than Capacity

If:

item > capacity

the item cannot fit into any bin.

Therefore, the input should be rejected as invalid.

Case 5: Multiple Items of Same Size

Duplicate item sizes are handled normally.

For example:

4, 4, 4, 4

can be distributed among the available bins according to their remaining capacity.

Case 6: All Items Fit in One Bin

If the sum of all items is less than or equal to the capacity, only one bin is required.

12. Advantages and Limitations

Advantages

1. Simple and easy to implement.
2. Faster than exact exponential approaches.
3. Sorting larger items first generally improves packing quality.
4. Every item is guaranteed to be considered.
5. Uses simple loops and conditional checks.
6. Works well for practical medium and large inputs.
7. Provides a theoretical approximation guarantee.

Limitations

1. It does not always produce the optimal packing.
2. The sorting step adds computational overhead.
3. The placement phase can take quadratic time in the worst case.
4. The result depends on the order in which equally sized items are processed.
5. Some input distributions can cause FFD to use more bins than the optimal solution.

13. Conclusion

- The First-Fit Decreasing algorithm provides an efficient approximation solution to the Bin Packing Problem.
- The algorithm first sorts items in decreasing order and then places each item into the first available bin with sufficient remaining capacity. If no existing bin can accommodate the item, a new bin is created.
- The use of sorting, loops, conditional checks, and a placement flag makes the algorithm simple, structured, and efficient. It handles important edge cases such as empty input, invalid item sizes, items equal to the bin capacity, and items larger than the capacity.
- Although FFD does not always guarantee the absolute minimum number of bins, it provides a strong practical solution with a known approximation bound and a significantly lower computational cost than exact optimization methods.
- Therefore, First-Fit Decreasing is an appropriate and effective approximation algorithm for the Bin Packing Problem.

SIMATS Online Lab

ctivity/attempt?assignment_id=1625

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

MOHAN KUMAR J
192424060RA

ASSESSMENT-CO5-AT2-Pseudocode Writing Task (Algorithm Design)

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1
Bin Packing Approximation Algorithm
100 marks

1/1 answered

Q1 Bin Packing Approximation Algorithm — 100 marks

Write a pseudocode for an approximation algorithm such as First-Fit Decreasing to pack items into bins of fixed capacity. Clearly define inputs (item sizes, bin capacity) and output (set of bins with assigned items). Design an algorithm that first orders items and then places each item into the first suitable bin. Use loops and conditional checks effectively for sorting and bin assignment. Ensure the pseudocode is well-structured, readable, and logically organized. Present all major steps clearly in sequence. Handle all items correctly and produce a valid approximate packing solution.

Your Code

```
def first_fit_decreasing(items, capacity):
    items_sorted = sorted(items, reverse=True)
    bins = []
    remaining = []
    for item in items_sorted:
        placed = False
        for i in range(len(bins)):
            if remaining[i] >= item:
                bins[i].append(item)
                remaining[i] -= item
                placed = True
                break
        if not placed:
            bins.append([item])
            remaining.append(capacity - item)
    return bins, remaining
def main():
    capacity = int(input())
    items_input = input()
    items = list(map(int, items_input.split()))
    bins, remaining = first_fit_decreasing(items, capacity)
    print("\nNumber of bins used:", len(bins))
    for i, b in enumerate(bins):
        print(f"Bin {i + 1}: items = {b}, remaining capacity = {remain
if __name__ == "__main__":
    main()
```

Input

```
10
2 5 4 7 1 3 8
```

Output

```
Bin 2: items = [7, 3],
remaining capacity = 0
Bin 3: items = [5, 4, 1],
remaining capacity = 0
```

Submitted